



GUÍA DE ARQUITECTURA

# Arquitectura Demeter

Cómo está construido el sistema de gestión de la granja porcícola Demeter — qué hace cada microservicio, cómo se comunican, y el patrón exacto a seguir para agregar uno nuevo.

---

**12**

MICROSERVICIOS

**8000–8011**

RANGO DE PUERTOS

**1**

CLAVE JWT COMPARTIDA

**SQLite**

UNA BASE POR SERVICIO

Python 3.13 · FastAPI · SQLAlchemy · Pydantic v2 · JWT compartido vía `demeter_core`

## CONTENIDO

01 Qué es Demeter

---

02 Mapa de servicios

---

03 Referencia funcional de cada servicio

---

04 Arquitectura general

---

05 Estructura estándar de un microservicio

---

06 Autenticación (JWT)

---

07 Cliente de IA generativa

---

08 Testing

---

09 Cómo agregar un microservicio nuevo

---

10 Deuda técnica conocida

---

## 01 Qué es Demeter

Demeter es un sistema de gestión para una granja porcícola, construido como una colección de microservicios FastAPI independientes. Cada servicio es responsable de un dominio del negocio (corrales, inventario de cerdos, alimentación, sensores IoT, usuarios, notificaciones, IA) y se comunica con los demás por HTTP — nunca compartiendo tablas de base de datos.

**Stack:** Python 3.13, FastAPI, SQLAlchemy + SQLite, Pydantic v2, `slowapi` (rate limiting), `python-jose` (JWT), `httpx` / `requests` entre servicios, `scikit-learn` / `joblib` para los modelos de Machine Learning clásico, y un cliente propio para modelos de IA generativa (Claude / OpenAI / vLLM).

`core_compartido/demeter_core` es una librería instalada en modo editable ( `pip install -e core_compartido` ) que varios servicios reutilizan: enums de dominio, esquemas Pydantic compartidos, el cliente de IA generativa, y la verificación de JWT.

## 02 Mapa de servicios

Cada servicio ocupa un puerto fijo. Los de `dominio` gestionan datos del negocio; los de `auth` emiten o verifican identidad; los de `IA` corren modelos (clásicos o generativos); `infra` son piezas de soporte, no expuestas por el gateway.

| PUERTO | SERVICIO          | TIPO    | RESPONSABILIDAD                                                        |
|--------|-------------------|---------|------------------------------------------------------------------------|
| 8000   | alimentacion      | dominio | Registro <b>agregado por corral</b> de raciones de comida (lote)       |
| 8001   | corrales          | dominio | Alta y gestión de corrales (capacidad, área, etapa productiva)         |
| 8002   | alimentacion_ia   | IA      | Registro <b>individual por RFID</b> de comederos + predicción de dieta |
| 8003   | inventario        | dominio | Inventario de cerdos: alta, camadas, ubicación                         |
| 8004   | iot               | dominio | Ingesta de lecturas de sensores ambientales                            |
| 8005   | usuarios          | auth    | Registro, login, JWT (access + refresh), roles                         |
| 8006   | ia                | IA      | ML clásico: predicción de estrés calórico según clima                  |
| 8007   | ia_notificaciones | IA      | ML clásico: decide si y cómo notificar                                 |
| 8008   | clima_externo     | dominio | Clima satelital (Open-Meteo) + consulta a ia + historial               |
| 8009   | ordenes_trabajo   | dominio | Genera órdenes de trabajo (SOP) para alimento manual por lote          |
| 8010   | notificaciones    | dominio | Envío de alertas WhatsApp, redacción opcional vía IA generativa        |
| 8011   | ml_ws             | infra   | Reenvío/placeholder — menor madurez, no está en nginx                  |

## 03 Referencia funcional de cada servicio

Endpoints, reglas de negocio y modelo de datos de los 12 servicios, en orden de puerto.  = requiere `Depends(obtener_usuario_actual)` o `requiere_rol(...)`; sin candado = abierto (lectura pública, o telemetría de dispositivo).

### **alimentacion** :8000 raciones por corral

| MÉTODO | RUTA               | AUTH                                                                              | QUÉ HACE                                  |
|--------|--------------------|-----------------------------------------------------------------------------------|-------------------------------------------|
| POST   | /alimentacion/     |  | Registra una ración entregada a un corral |
| GET    | /alimentacion/     | —                                                                                 | Lista registros, filtro opcional ?corral= |
| GET    | /alimentacion/{id} | —                                                                                 | Detalle de un registro                    |
| PATCH  | /alimentacion/{id} |  | Corrige un registro                       |
| DELETE | /alimentacion/{id} |  | Elimina un registro                       |

- **Regla:** valida contra `inventario` que el corral tenga cerdos antes de registrar o reasignar.
- **Modelo** (`registros_alimentacion`): `corral`, `tipo_alimento`, `cantidad_kg`, `fecha_hora`.

### **corrales** :8001 gestión de corrales

| MÉTODO | RUTA               | AUTH                                                                                | QUÉ HACE                                            |
|--------|--------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------|
| POST   | /corrales/         |  | Crea un corral; calcula area_m2                     |
| GET    | /corrales/         | —                                                                                   | Lista todos los corrales                            |
| GET    | /corrales/{nombre} | —                                                                                   | Busca un corral por nombre                          |
| PATCH  | /corrales/{id}     |  | Actualiza; recalcula el área si cambian las medidas |
| DELETE | /corrales/{id}     |  | Elimina un corral                                   |

- **Regla:** el área nunca se recibe del cliente, siempre se calcula en el servidor.
- **Modelo** (`corrales`): `nombre` (único), `capacidad_maxima`, `ancho_m`, `largo_m`, `area_m2`, `etapa`.

### **alimentacion\_ia** :8002 comedero RFID + IA nutricional

| MÉTODO | RUTA                           | AUTH       | QUÉ HACE                                                |
|--------|--------------------------------|------------|---------------------------------------------------------|
| POST   | /alimentacion/predecir_dieta   | —          | Modelo ML recomienda ración según edad/peso/temperatura |
| POST   | /alimentacion/registrar_comida | telemetría | Evento de comedero: cerdo (RFID), corral, cantidad      |
| GET    | /alimentacion/historial/{rfid} | —          | Historial de comidas de un cerdo                        |

- **Regla:** valida contra `inventario` que el RFID pertenezca a un cerdo asignado a ese corral.
- **Regla:** agrega advertencia de ración reducida si `temperatura_c > 28`.
- **Modelo** (base propia `alimentacion_ia.db`): `id_cerdo_rfid`, `corral`, `racion_servida_kg`, `fecha_hora`.

## inventario :8003 inventario de cerdos

| MÉTODO | RUTA                              | AUTH | QUÉ HACE                      |
|--------|-----------------------------------|------|-------------------------------|
| POST   | /cerdos/                          |      | Registra un cerdo nuevo       |
| GET    | /cerdos/                          | —    | Lista cerdos, filtro ?corral= |
| GET    | /cerdos/{id}                      | —    | Detalle de un cerdo           |
| PATCH  | /cerdos/{id}                      |      | Actualización parcial         |
| DELETE | /cerdos/{id}                      |      | Elimina un cerdo              |
| GET    | /cerdos/madre/{etiqueta}/lechones | —    | Camada de una cerda madre     |

- **Regla:** valida contra `corrales` que la etapa del cerdo coincida con la del corral y que no haya hacinamiento.
- **Modelo** (`cerdos`): `etiqueta` (única), `raza?`, `fecha_nacimiento`, `etapa`, `madre_etiqueta?`, `peso_kg`, `corral`.

## iot :8004 sensores ambientales

| MÉTODO | RUTA                 | AUTH       | QUÉ HACE                              |
|--------|----------------------|------------|---------------------------------------|
| POST   | /iot/                | telemetría | Recibe un ping de temperatura/humedad |
| GET    | /iot/corral/{nombre} | —          | Últimas N lecturas (?limite=10)       |

- **Regla:** valida contra `corrales` que el corral exista.
- **Modelo** (`lecturas_iot`): `corral`, `temperatura_c`, `humedad_pct`, `fecha_hora`. Cliente de referencia: `sensor_virtual.py`.

## usuarios :8005 autenticación

| MÉTODO | RUTA              | AUTH     | QUÉ HACE                                     |
|--------|-------------------|----------|----------------------------------------------|
| POST   | /usuarios/        | registro | Crea un usuario, hashea password con bcrypt  |
| POST   | /usuarios/login   | —        | Emite access_token (1h) + refresh_token (7d) |
| GET    | /usuarios/        | admin    | Lista todos los usuarios                     |
| POST   | /usuarios/refresh | refresh  | Rota el refresh token                        |

- **Regla:** se guarda el hash del refresh token, no el valor en texto plano.
- **Modelo** (`usuarios`): `username` (único), `password_hash`, `rol`, `activo`, `refresh_token_hash`.

## ia :8006 predicción de estrés calórico

| MÉTODO | RUTA      | AUTH | QUÉ HACE                                    |
|--------|-----------|------|---------------------------------------------|
| POST   | /predecir | —    | Clasifica riesgo (0/1/2) desde clima y peso |

- Sin base de datos propia: modelo `.pk1` en memoria; responde 503 si no existe.

### ia\_notificaciones :8007 motor de decisión de alertas

| MÉTODO | RUTA                 | AUTH | QUÉ HACE                                  |
|--------|----------------------|------|-------------------------------------------|
| POST   | /evaluar-y-notificar | —    | Decide: omitir, email, o WhatsApp urgente |

- **Regla:** si decide WhatsApp urgente, despacha automáticamente a `notificaciones:8010/notificar`.

### clima\_externo :8008 clima satelital + diagnóstico

| MÉTODO | RUTA                               | AUTH | QUÉ HACE                                    |
|--------|------------------------------------|------|---------------------------------------------|
| POST   | /clima_externo/consultar_y_guardar | —    | Clima por GPS + bienestar de ia + historial |
| GET    | /clima_externo/historial/{granja}  | —    | Historial de una granja                     |

- **Regla:** si `ia` no responde, guarda igual con `estado_riesgo="DESCONOCIDO"`.

### ordenes\_trabajo :8009 órdenes de trabajo (SOP)

| MÉTODO | RUTA                        | AUTH                                                                               | QUÉ HACE                                 |
|--------|-----------------------------|------------------------------------------------------------------------------------|------------------------------------------|
| POST   | /ordenes/generar            |   | Calcula la fórmula del lote según el SOP |
| PUT    | /ordenes/completar/{codigo} |  | El operario confirma la preparación      |
| GET    | /ordenes/pendientes         | —                                                                                  | Órdenes en estado PENDIENTE              |

- **Regla:** la ración se reduce por estrés calórico y se corta a cero si `peso_promedio_kg >= 110` (pre-faena).
- **Modelo** (`ordenes_trabajo`): `codigo_orden` (único), `corral_objetivo`, `total_raciones_kg`, `receta_json`, `instrucciones_json`, `estado`.

### notificaciones :8010 despacho de alertas WhatsApp

| MÉTODO | RUTA                  | AUTH | QUÉ HACE                                           |
|--------|-----------------------|------|----------------------------------------------------|
| POST   | /evaluar_notificacion | —    | Modelo ML decide si es buen momento para notificar |
| POST   | /notificar            | —    | Despacho directo, usado por ia_notificaciones      |

- **Regla:** texto redactado por IA generativa si `AI_MESSAGES_ENABLED=true`, con plantilla fija de respaldo.

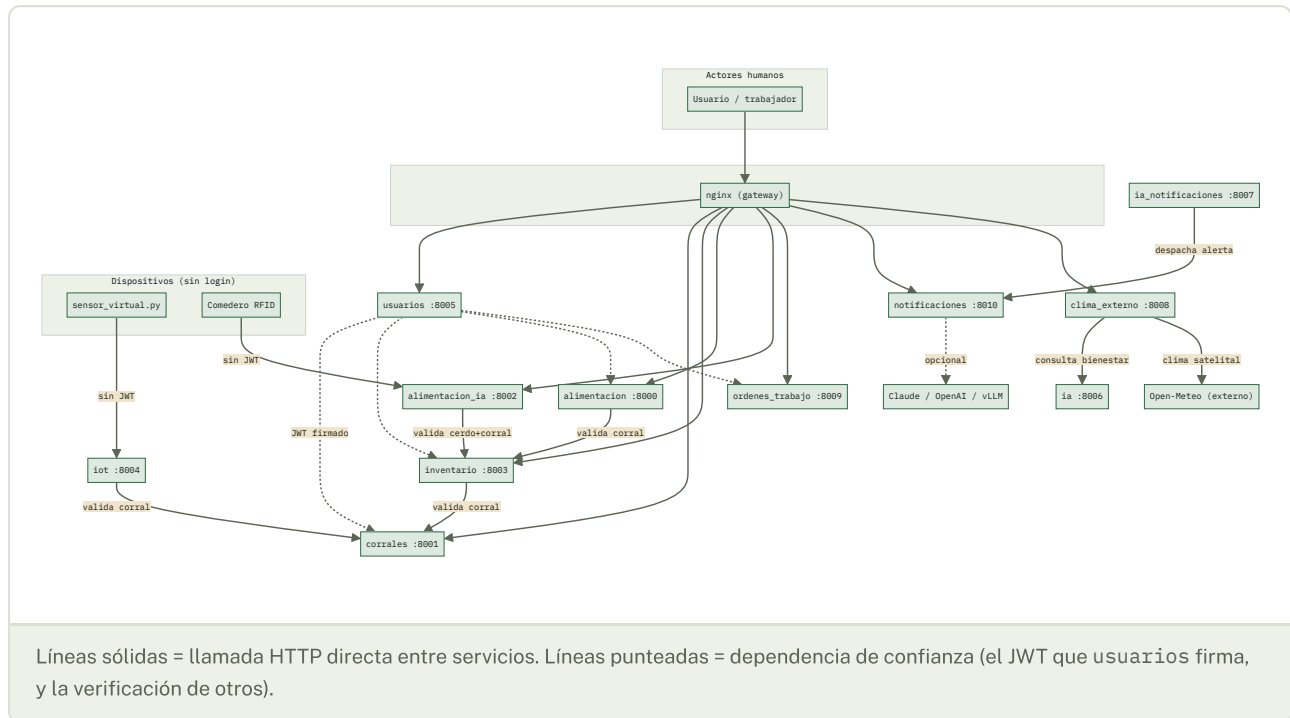
### ml\_ws :8011 reenvío (infra, sin madurez)

| MÉTODO | RUTA               | AUTH | QUÉ HACE                                |
|--------|--------------------|------|-----------------------------------------|
| POST   | /send-notification | —    | Reenvía un payload hacia notificaciones |

- Sin capas `db` / `schemas`; no está en `nginx.conf`. Tratar como scratch.

## 04 Arquitectura general

Cada servicio tiene su propia base de datos SQLite — no hay tablas compartidas. Si necesita datos de otro, lo consulta por HTTP. Las URLs de otros servicios se leen de variables de entorno con un valor por defecto correcto, nunca hardcodeadas sin fallback.



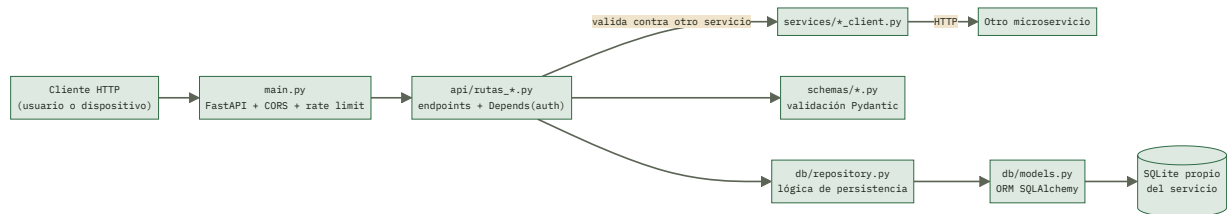
### Reglas de comunicación:

- El JWT se aplica solo a **acciones humanas que modifican datos** (crear/editar/borrar corrales, cerdos, registros, órdenes).
- Los endpoints de **telemetría de dispositivo** (sensor IoT, comedero RFID) quedan sin JWT a propósito: un sensor no puede iniciar sesión — eso es un problema distinto (credenciales de dispositivo) todavía sin resolver.
- Las lecturas (**GET**) quedan abiertas: otros servicios las consultan para validaciones cruzadas sin credenciales de usuario.

## 05 Estructura estándar de un microservicio

Todo microservicio nuevo sigue esta misma plantilla de carpetas:

```
microservicio_nuevo/
├── main.py          # Bootstrap: FastAPI(), CORS, rate limit, router, health check
├── api/
│   ├── __init__.py
│   └── rutas_nuevo.py  # Endpoints, validación cruzada, Depends(auth) si aplica
├── db/
│   ├── __init__.py
│   ├── database.py    # engine, SessionLocal, get_db() — anclado a __file__
│   └── models.py      # Modelos SQLAlchemy (ORM)
├── schemas/
│   ├── __init__.py
│   └── nuevo.py       # Pydantic: Create / Response / Update
├── services/        # Solo si este servicio llama a otros
│   ├── __init__.py
│   └── otro_client.py  # Cliente HTTP hacia otro microservicio
```



Las rutas nunca llaman a `db.query(...)` directamente — delegan siempre a `db/repository.py`. Saltarse esa capa ya causó una regresión real en este proyecto.

### Capas y su responsabilidad

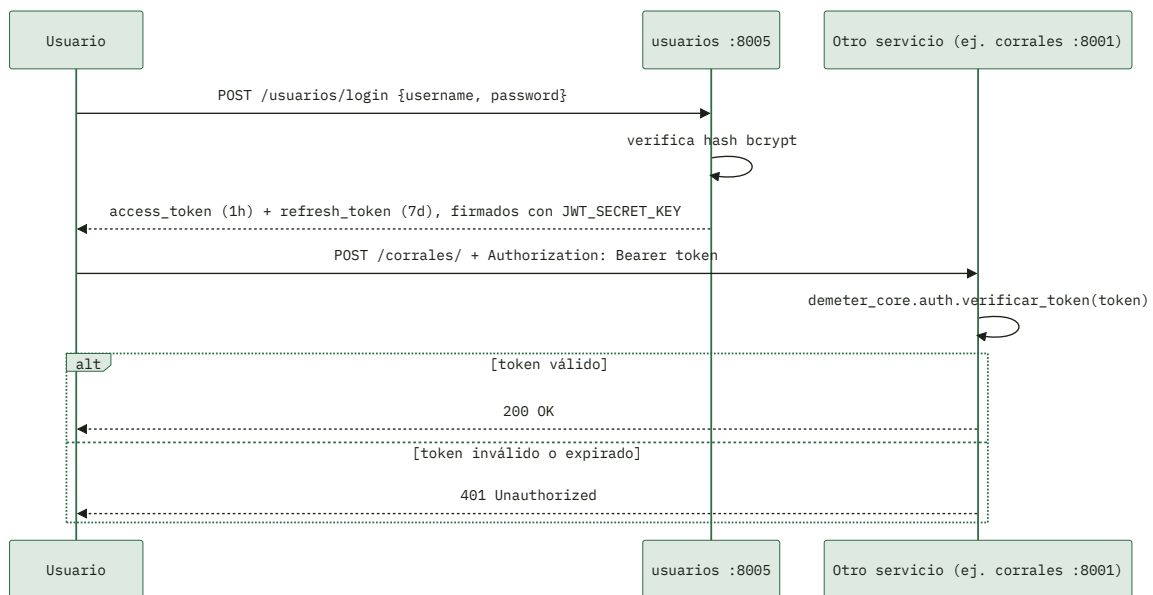
- **main.py** — solo bootstrap: crea la app, registra CORS y rate limiting, llama `Base.metadata.create_all(bind=engine)`, incluye el router, expone `GET /` como health check.
- **api/rutas\_\*.py** — los endpoints. Validan contra otros servicios vía `services/*_client.py`, delegan persistencia a `db/repository.py`, y aplican `Depends(obtener_usuario_actual)` en mutaciones humanas.
- **db/repository.py** — funciones puras que reciben una `Session` y hacen el CRUD.
- **db/database.py** — `engine` con ruta anclada a `os.path.dirname(os.path.abspath(__file__))`, nunca `sqlite:///./algo.db` relativo — eso colisionó una vez entre dos servicios (`alimentacion.db` vs. `alimentacion_ia.db`).
- **schemas/\*.py** — `Create` (entrada), `Response` (salida, nunca contraseñas/hashes), `Update` (todo `Optional`).

**Imports:** siempre relativos (`.db`, `..schemas`, `..services`), nunca `from db import models`. Mezclar los dos estilos fue la causa real de que 6 de 12 servicios no arrancaran ni corrieran sus tests.

El servicio se ejecuta siempre **desde la raíz del repositorio**:

```
python -m microservicio_nuevo.main  
# o equivalentemente:  
uvicorn microservicio_nuevo.main:app --reload
```

## 06 Autenticación (JWT)



`usuarios` es el único que emite tokens; `demeter_core.auth` es el único lugar que los verifica — cualquier servicio lo importa igual.

Los dos deben compartir el mismo `JWT_SECRET_KEY` (variable de entorno). Si no se define en ningún lado, todos caen al mismo valor de desarrollo por defecto — funciona igual en local sin configurar nada, pero antes de un despliegue real hay que fijarlo explícitamente e idéntico en todos los servicios.

### Patrón para proteger un endpoint nuevo

```
from demeter_core.auth import obtener_usuario_actual, requiere_rol

@router.post("/")
def crear_algo(datos: AlgoCreate, db: Session = Depends(get_db),
              _usuario=Depends(obtener_usuario_actual)):
    ...

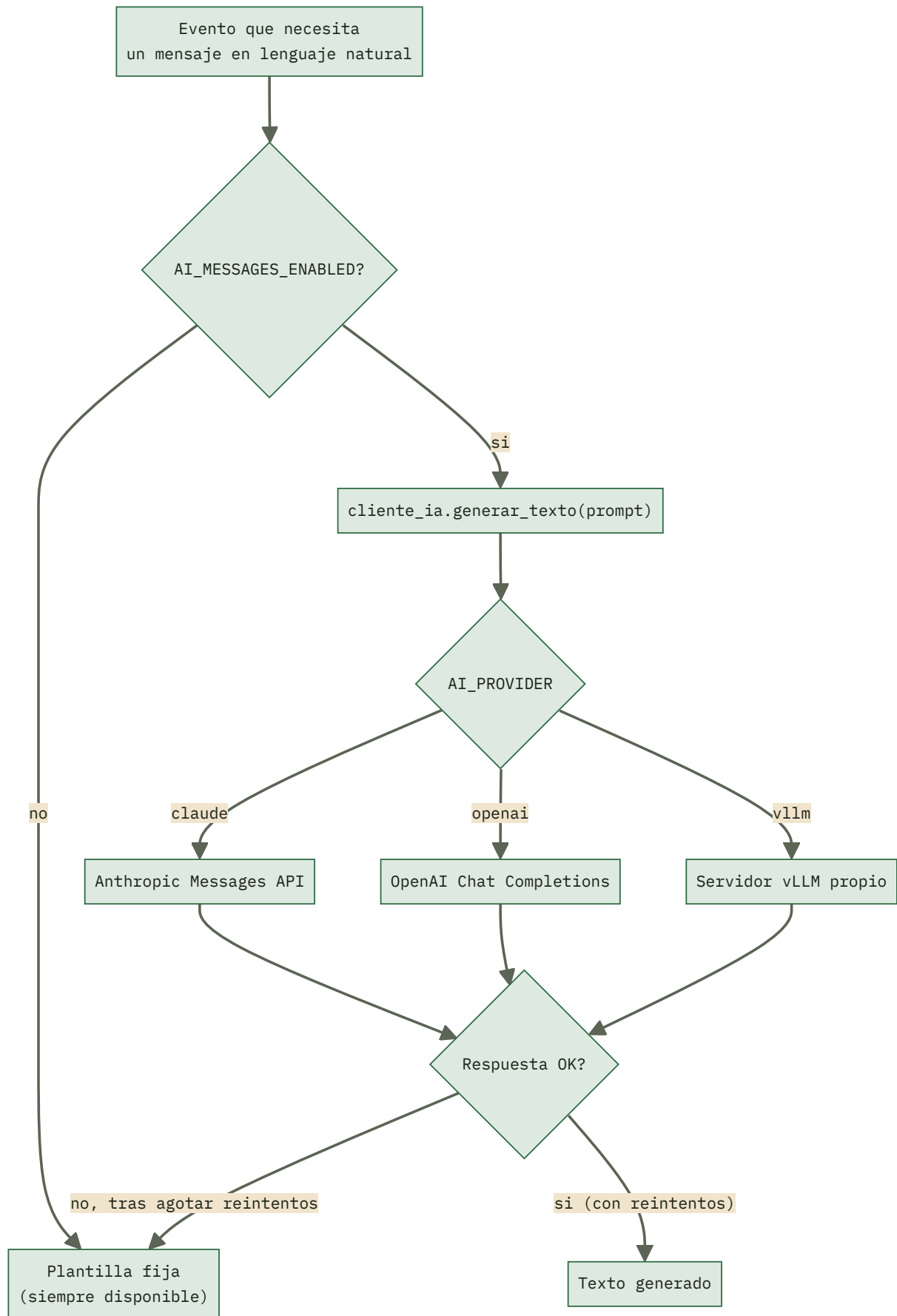
@router.get("/admin-only")
def listar_todo(_usuario=Depends(requiere_rol("administrador"))):
    ...
```

**Cuándo no proteger:** si el emisor real es un dispositivo (sensor, comedero automático) y no una persona, no agregues `Depends(obtener_usuario_actual)` — el dispositivo no puede loguearse con este sistema. Documenta el endpoint como abierto y deja la autenticación de dispositivos como una decisión aparte.

## 07 Cliente de IA generativa

---

`demeter_core.ai_client` soporta Claude, OpenAI y vLLM vía HTTP puro ( `httpx` ), sin SDKs adicionales — `ClienteVLLM` reutiliza el formato de `ClienteOpenAI` porque vLLM expone una API compatible.



El envío de una alerta nunca depende de que el proveedor externo esté disponible: siempre hay una plantilla fija de respaldo.

- Reintentos con backoff exponencial ante 429/5xx/timeout; si se agotan, levanta `AIClientError`.
- El cliente se crea **una sola vez** al arrancar el servicio (en el `lifespan` de FastAPI) y se reutiliza para todas las peticiones.
- Ejemplo real de uso: `microservicio_notificaciones/services/whatsapp_client.py`.

## 08 Testing

---

- **Aislamiento de base de datos:** cada archivo de test usa SQLite en memoria (`sqlite:///memory:` + `StaticPool`) y sobrescribe `get_db` vía `app.dependency_overrides`.
- **Mocking de llamadas a otros servicios:** se mockea la función del cliente, nunca se depende de que el otro servicio esté corriendo de verdad.
- `tests/conftest.py` expone `header_auth(rol=...)` para generar un JWT de prueba sin loguearse de verdad.

```
# Ejemplo mínimo para un endpoint protegido
def test_crear_algo_sin_token_devuelve_401():
    respuesta = client.post("/algo/", json={...})
    assert respuesta.status_code == 401

def test_crear_algo_con_token_devuelve_200():
    respuesta = client.post("/algo/", json={...}, headers=header_auth())
    assert respuesta.status_code == 200
```

Correr toda la suite desde la raíz: `pytest` (usa `pytest.ini`, que ya trae `asyncio_mode = auto`).

## 09 Cómo agregar un microservicio nuevo

|    |                                                    |                                 |                                                                                |                                |                                                                       |
|----|----------------------------------------------------|---------------------------------|--------------------------------------------------------------------------------|--------------------------------|-----------------------------------------------------------------------|
| 1  | Crear                                              | microservicio_nuevo/            | con la estructura de la sección 05 — cada subcarpeta con su                    | __init__.py                    | .                                                                     |
| 2  |                                                    | db/database.py                  | : engine anclado a                                                             | __file__                       | (copiar el patrón de microservicio_iot/db/database.py ).              |
| 3  | Definir los modelos ORM en                         | db/models.py                    | y los schemas Pydantic en                                                      | schemas/                       | .                                                                     |
| 4  | Escribir las rutas:                                | Depends(obtener_usuario_actual) | /                                                                              | requiere_rol(...)              | en mutaciones humanas; sin proteger las que vengan de un dispositivo. |
| 5  |                                                    | main.py                         | : copiar el bootstrap de un servicio ya migrado (ej.                           | microservicio_corrales/main.py | ) — CORS con allow_credentials=False , rate limit con                 |
| 6  | Elegir un puerto libre (sección 02) y usarlo en el | if __name__ == "__main__"       | .                                                                              |                                |                                                                       |
| 7  | Agregar la ruta correspondiente en                 | nginx/nginx.conf                | .                                                                              |                                |                                                                       |
| 8  | Si llama a otro servicio, crear                    | services/otro_client.py         | leyendo la URL de una variable de entorno con el puerto correcto como default. |                                |                                                                       |
| 9  | Escribir tests en                                  | tests/test_nuevo_service.py     | siguiendo el patrón de la sección 08.                                          |                                |                                                                       |
| 10 | Agregar dependencias nuevas a                      | requirements.txt                | (y a                                                                           | requirements-dev.txt           | si son solo para tests).                                              |
| 11 | Correr                                             | pytest                          | completo antes de terminar — no debe bajar el conteo de tests que pasan.       |                                |                                                                       |

## 10 Deuda técnica conocida

---

Para no repetirla ni sorprenderse:

- `microservicio_ml_ws` es un servicio placeholder/de scratch, no está en `nginx.conf`.
- Los scripts `entrenar_*.py` y `prueba_*.py` dentro de `notificaciones` e `ia` son herramientas de desarrollo, no parte de la app desplegable.
- La autenticación de dispositivos (sensor IoT, comedero RFID) sigue sin resolver — esos endpoints están abiertos a propósito, no por descuido.
- `.github/workflows/ci.yml` no construye ni testea todos los servicios; si agregas uno nuevo, sería buen momento para arreglar eso también.